CS 4400: Artificial Intelligence

Local search and optimization

Contents

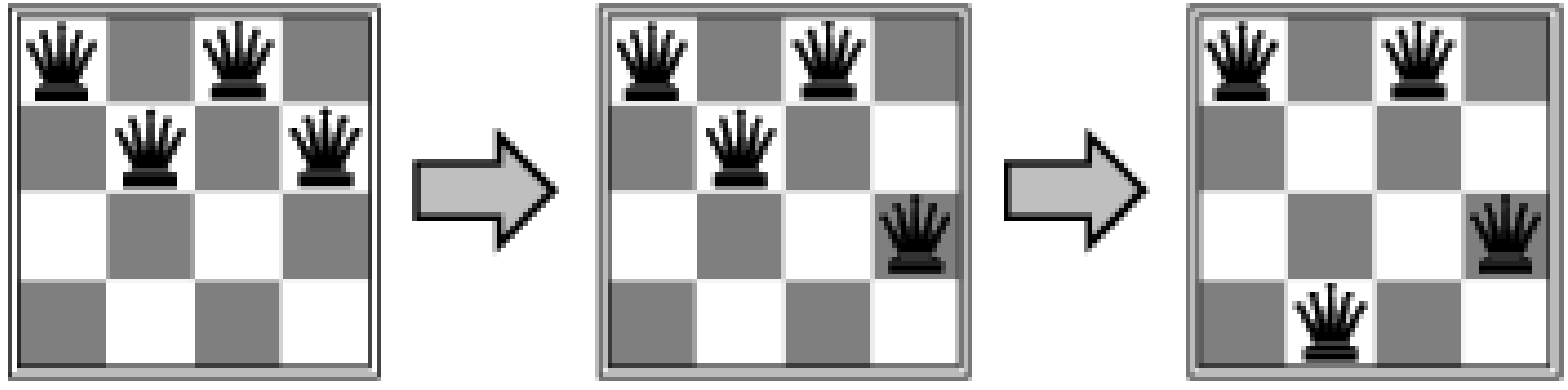
1. Local search
2. Hill climbing
3. Simulated annealing
4. Local beam search
5. Genetic algorithm

1/5. Local search

- Sometimes path doesn't matter, only the end goal does
  - N-queens
  - Integrated-circuit design
- Local search algorithms
  - Similar to greedy best-first search, but even simpler
  - Goal-finding algorithms that ignore paths
  - Can save memory (only remember current state)
  - Can often find solutions in large/infinite state space unsuitable for systematic search

Example: n-queens

- Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal

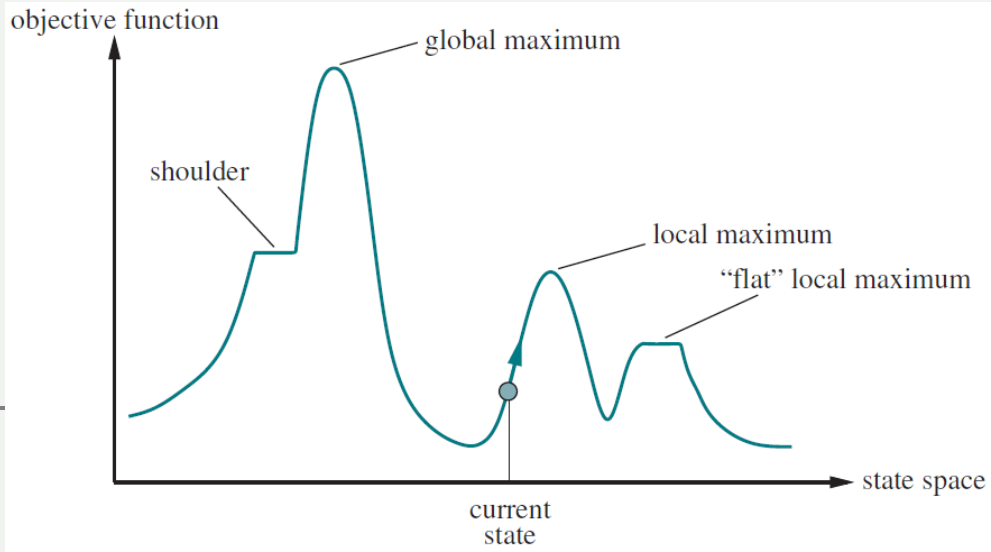


Note that a state cannot be an incomplete configuration with m<n queens

Local search

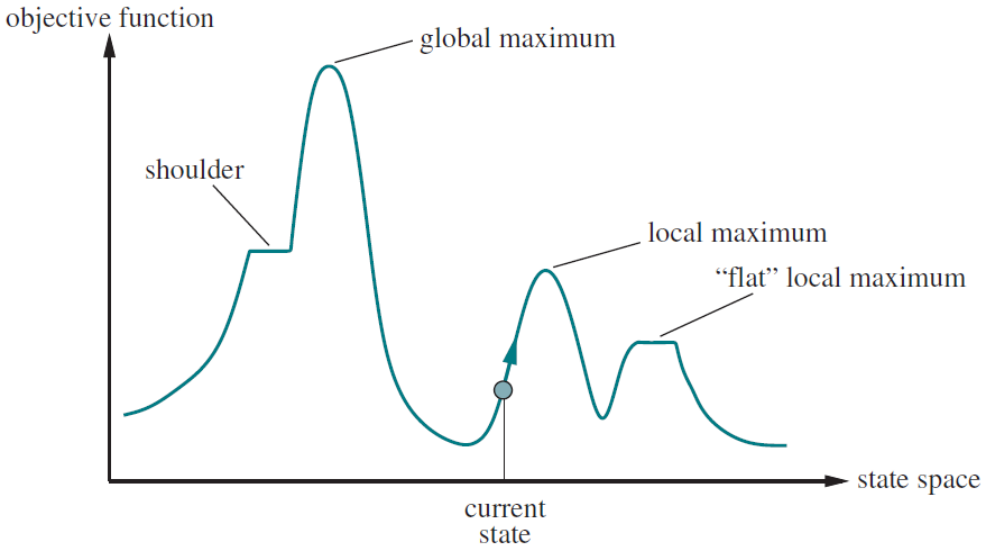
- Objective function
  - In tree/graph search: evaluation function, or negative of cost-to-go
  - Find a state with the maximum objective value
- State-space landscape
  - Plot of objective as a function of states
- Complete &

  Optimal?
  - No, in general



2/5. Hill-climbing

- Hill-climbing
  - One of the simplest local search
  - Continually move in the direction of increasing value
  - a.k.a. steepest-ascent/descent
  - Depending on initial state, can get stuck in local maxima

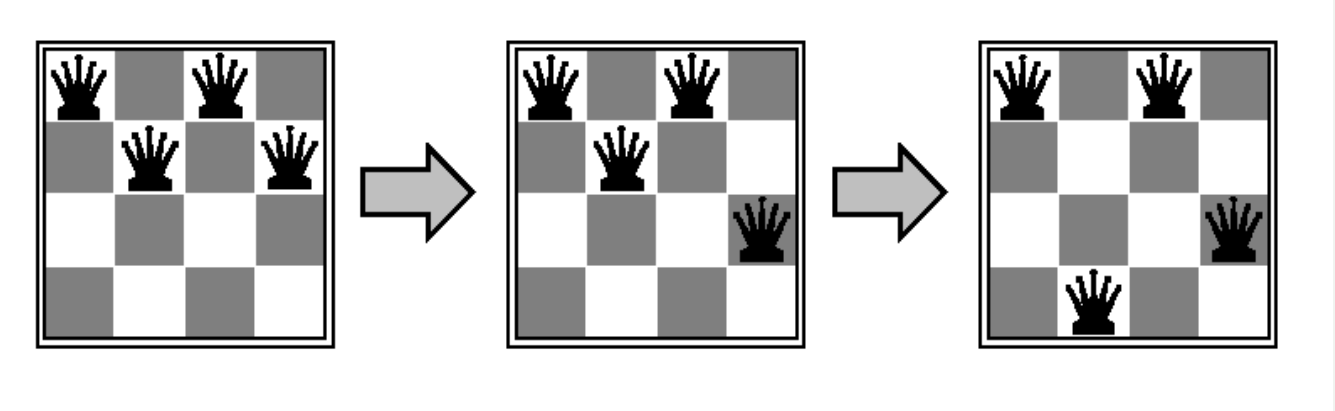


Hill-climbing

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor state of current   // random choice
                                                                 // to break a tile
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

- Start with a initial state
- Loop: improve by moving to neighboring states
- Don't keep the record of the past states
- *"Trying to find the top of Mount Everest in a thick fog while suffering from amnesia"*

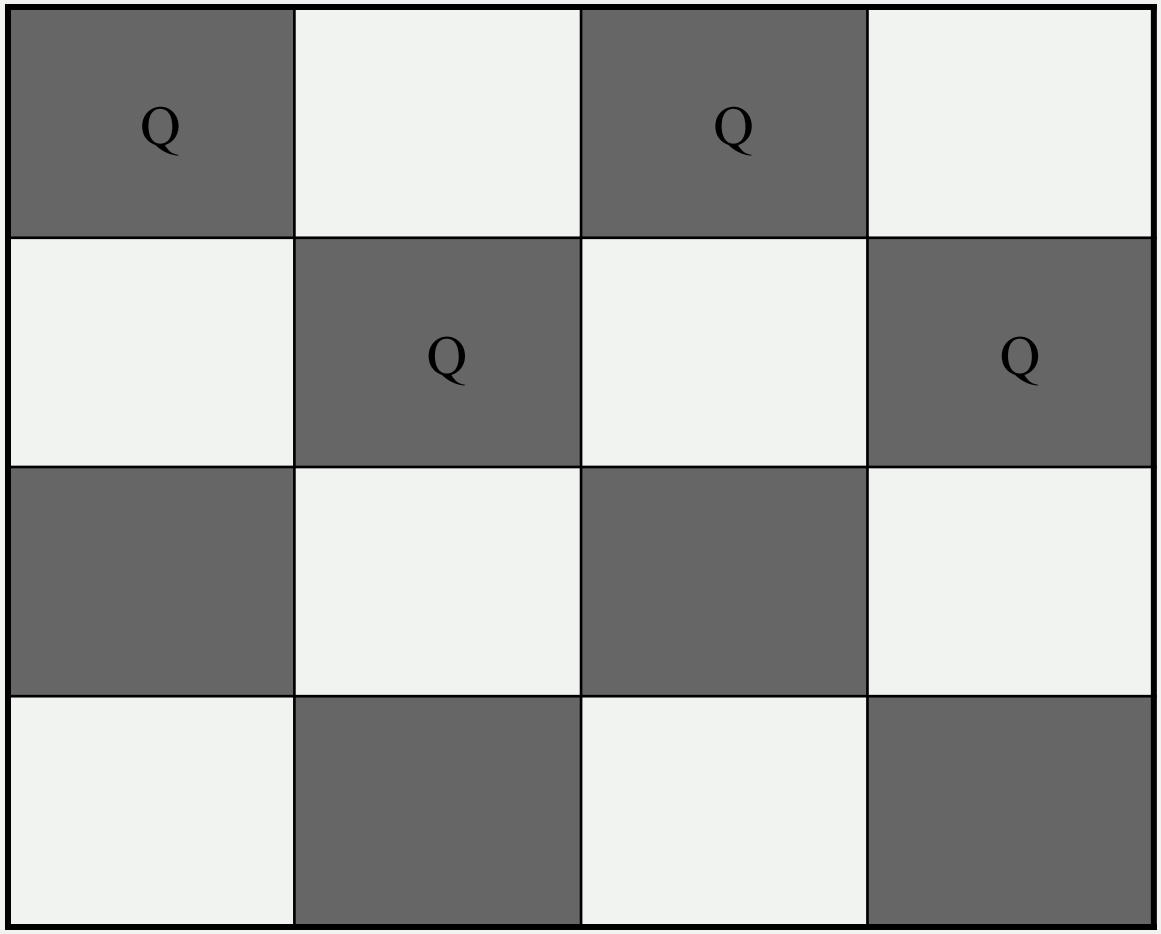
N-Queens problem



- State: a configuration of 4-queens
- Possible actions: choose one queen, and move the queen within the same column (12*)
- Cost : # of pairs of queens attacking each other (= -objective function)

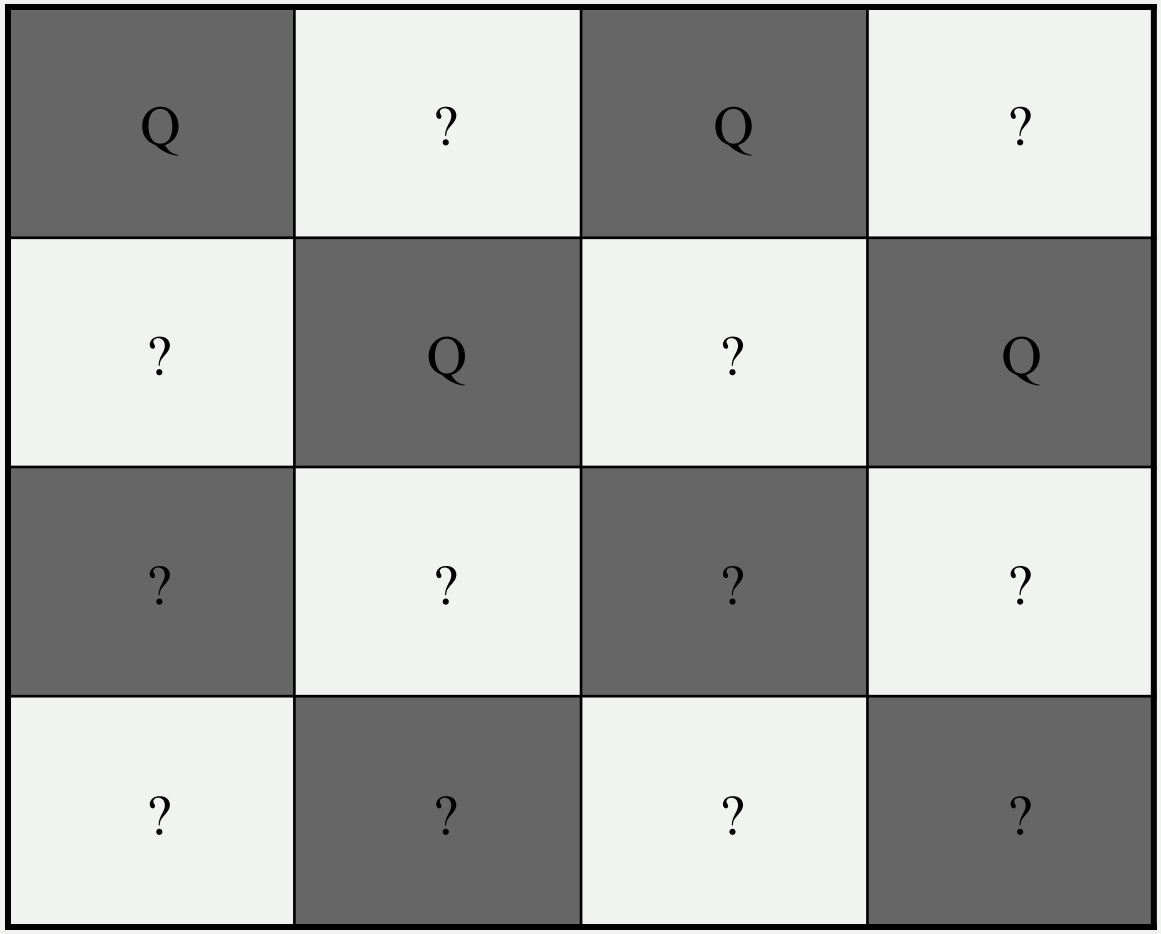
*Each column has 4 rows total, and since 1 row is already occupied by the queen, there are 3 alternative rows available for that queen.

N-Queens Hill Climbing



cost=?

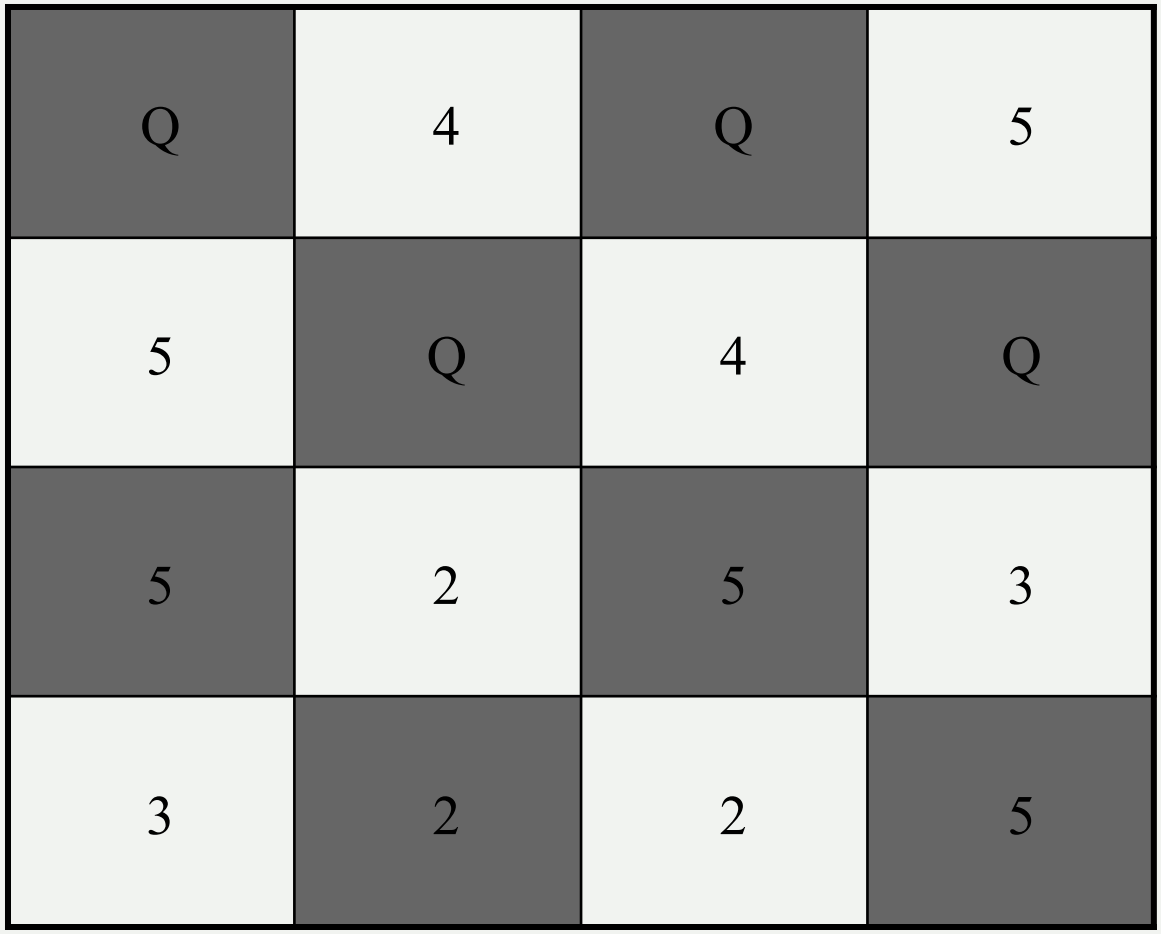
N-Queens Hill Climbing



cost=5 initially

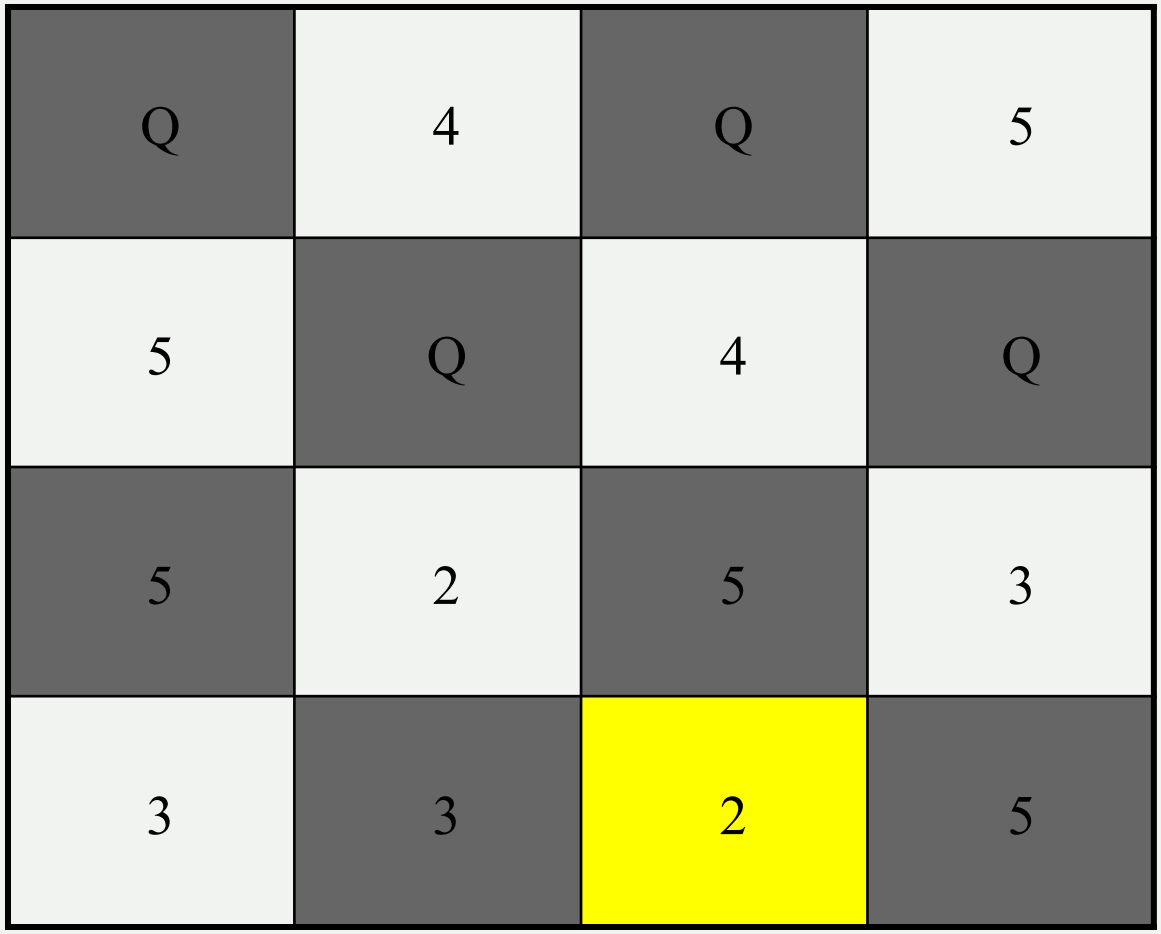
Cost after moving a queen?

N-Queens Hill Climbing



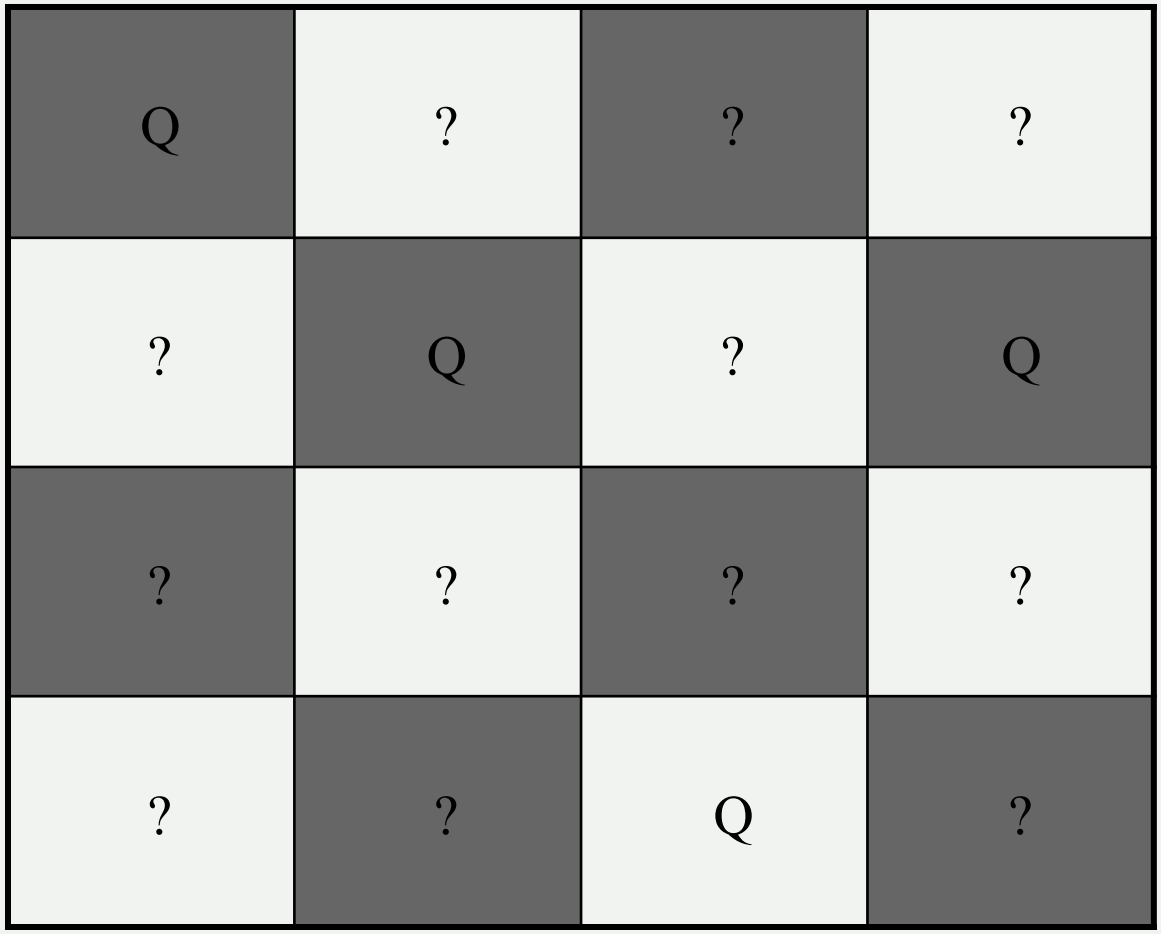
cost=5

N-Queens Hill Climbing



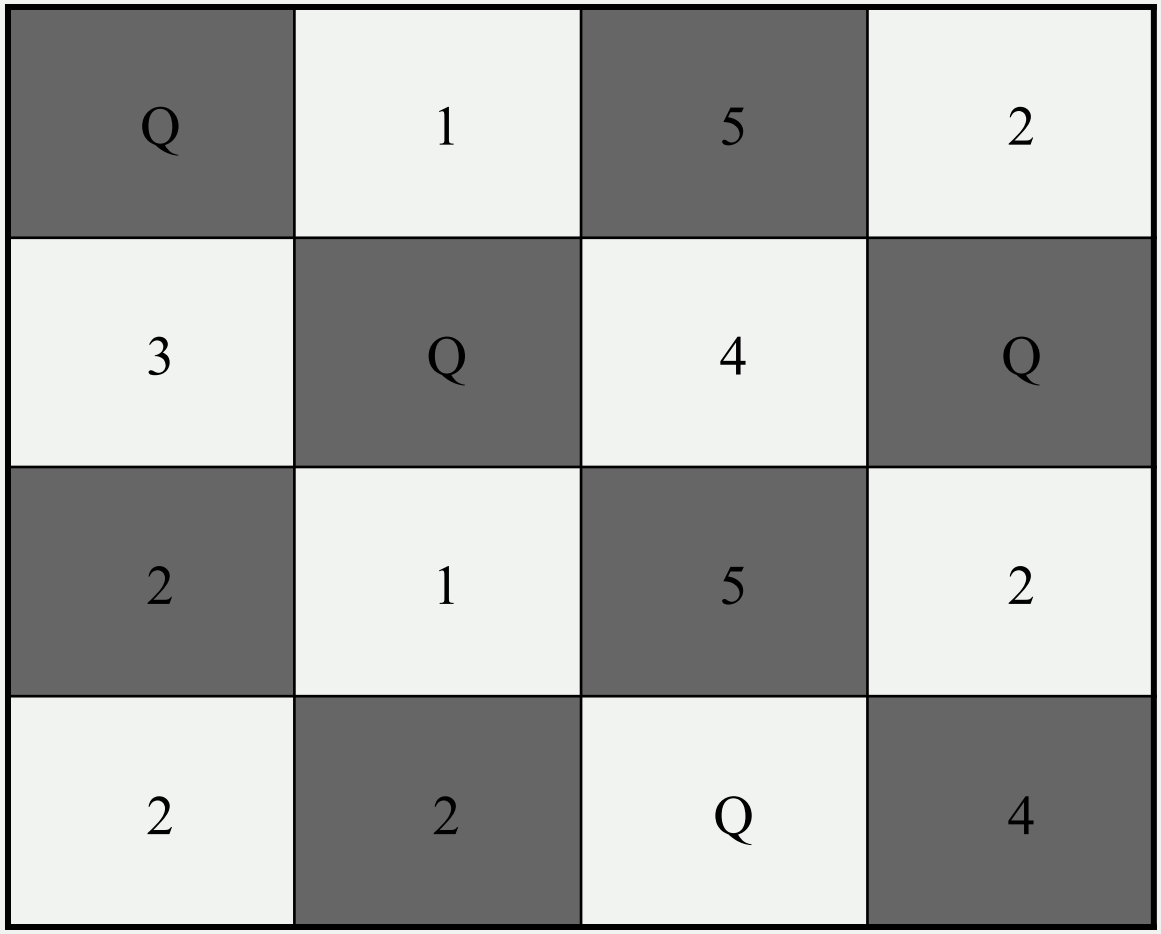
cost=5

N-Queens Hill Climbing



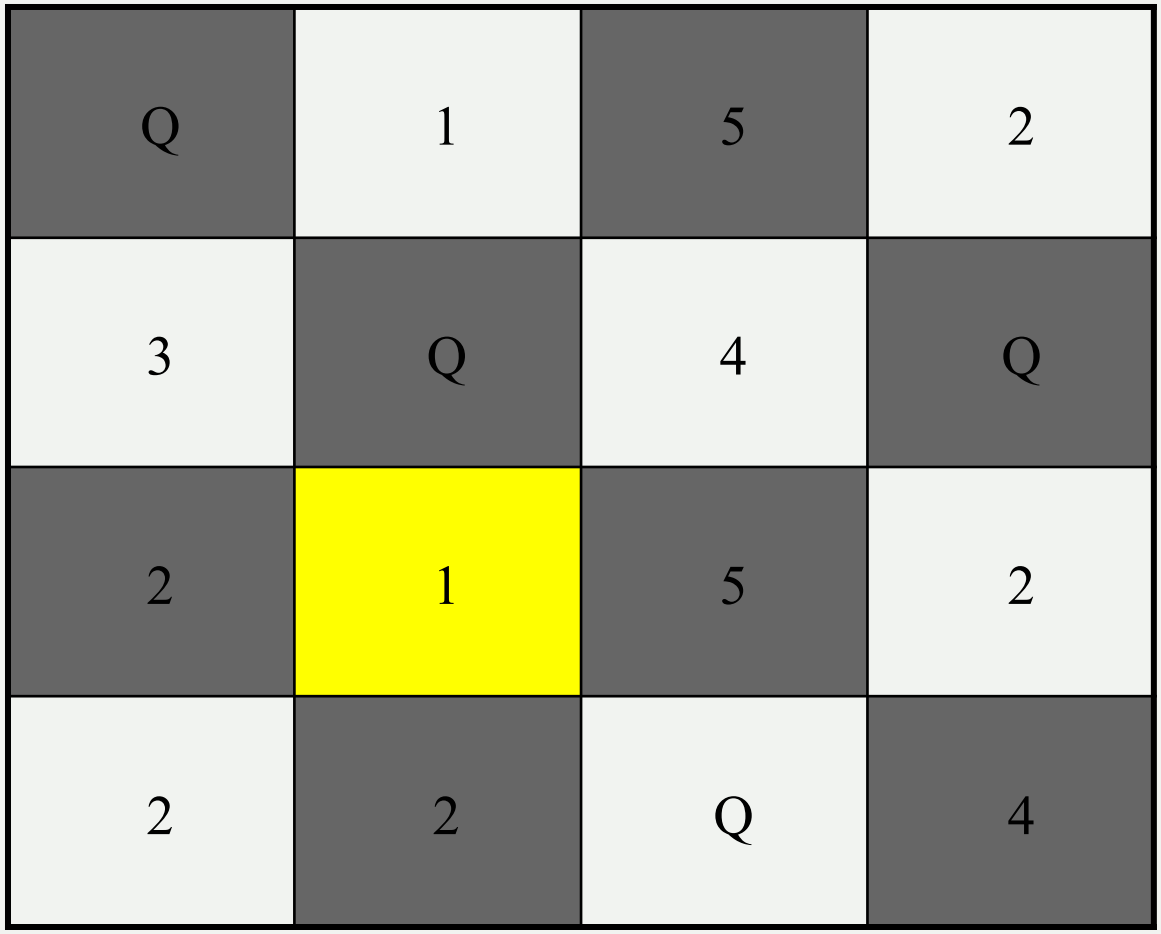
cost=2

N-Queens Hill Climbing



cost=2

N-Queens Hill Climbing



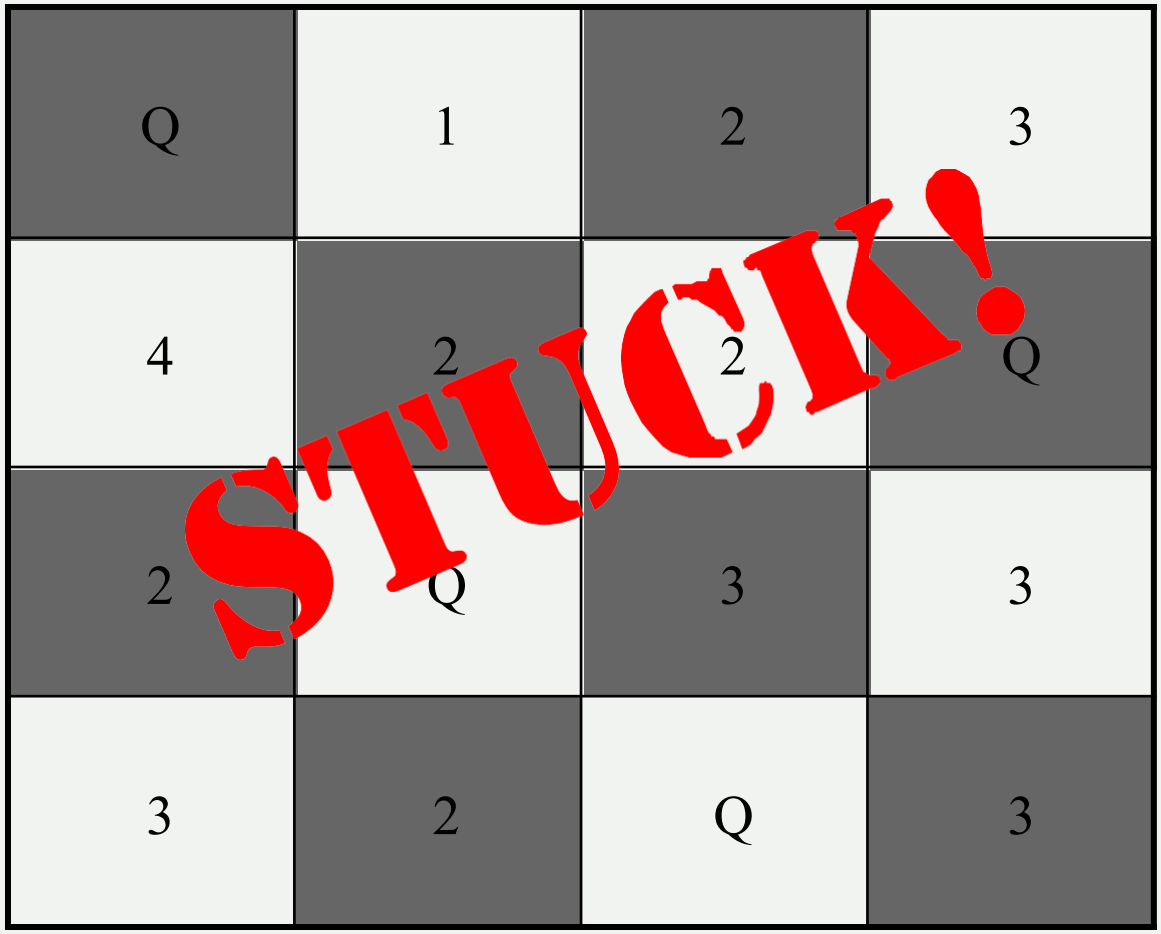
cost=2

N-Queens Hill Climbing

Q	1	2	3
4	2	2	Q
2	Q	3	3
3	2	Q	3

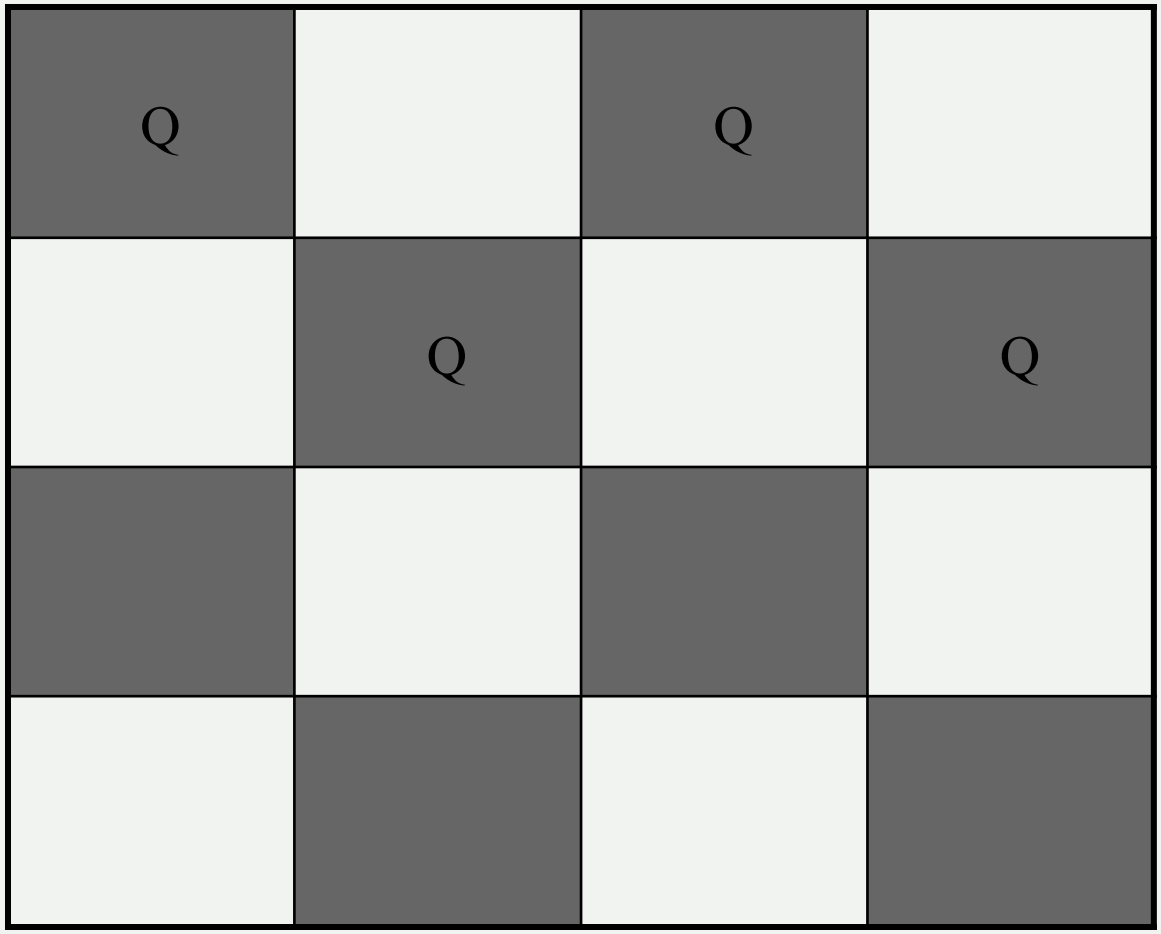
cost=1

N-Queens Hill Climbing



cost=1

N-Queens Hill Climbing – 2nd attempt



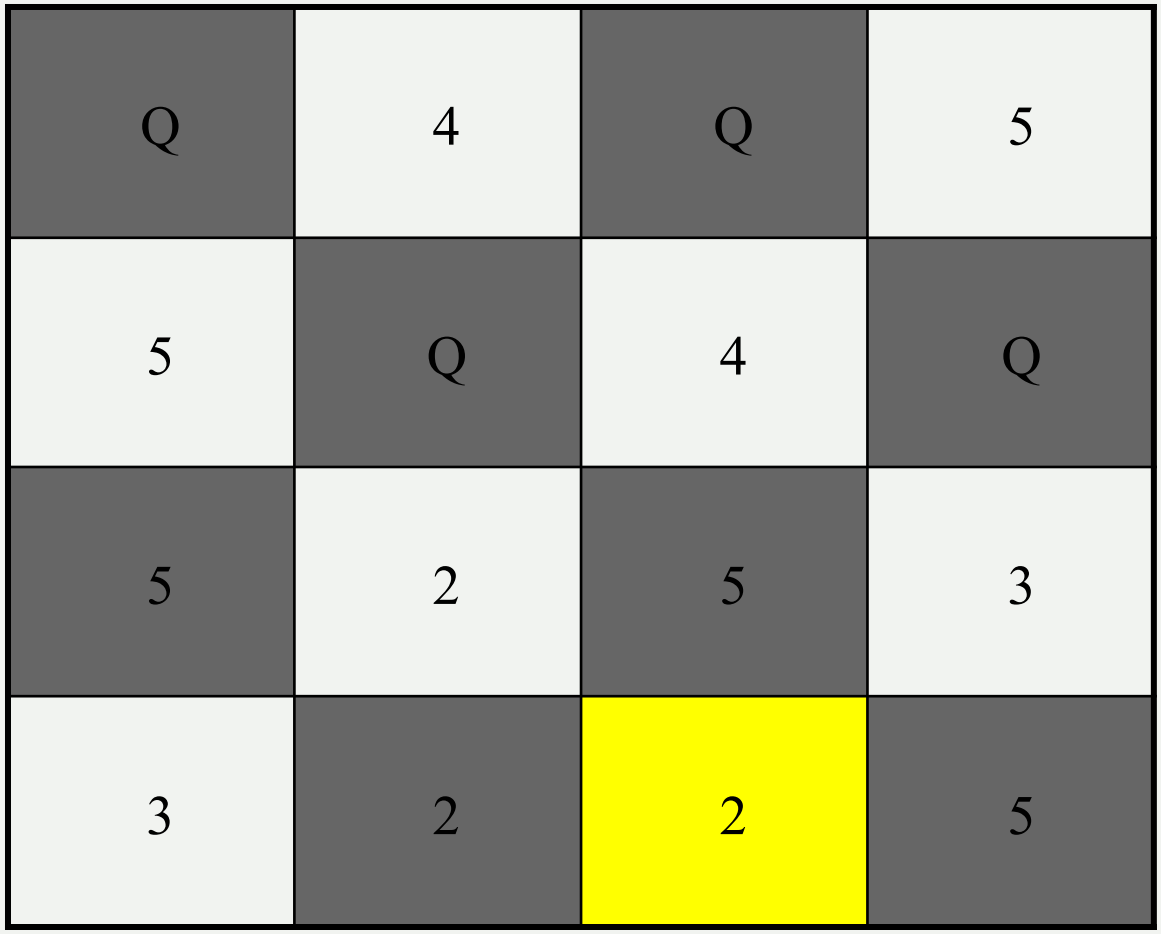
cost=5

N-Queens Hill Climbing – 2nd attempt

Q	4	Q	5
5	Q	4	Q
5	2	5	3
3	2	2	5

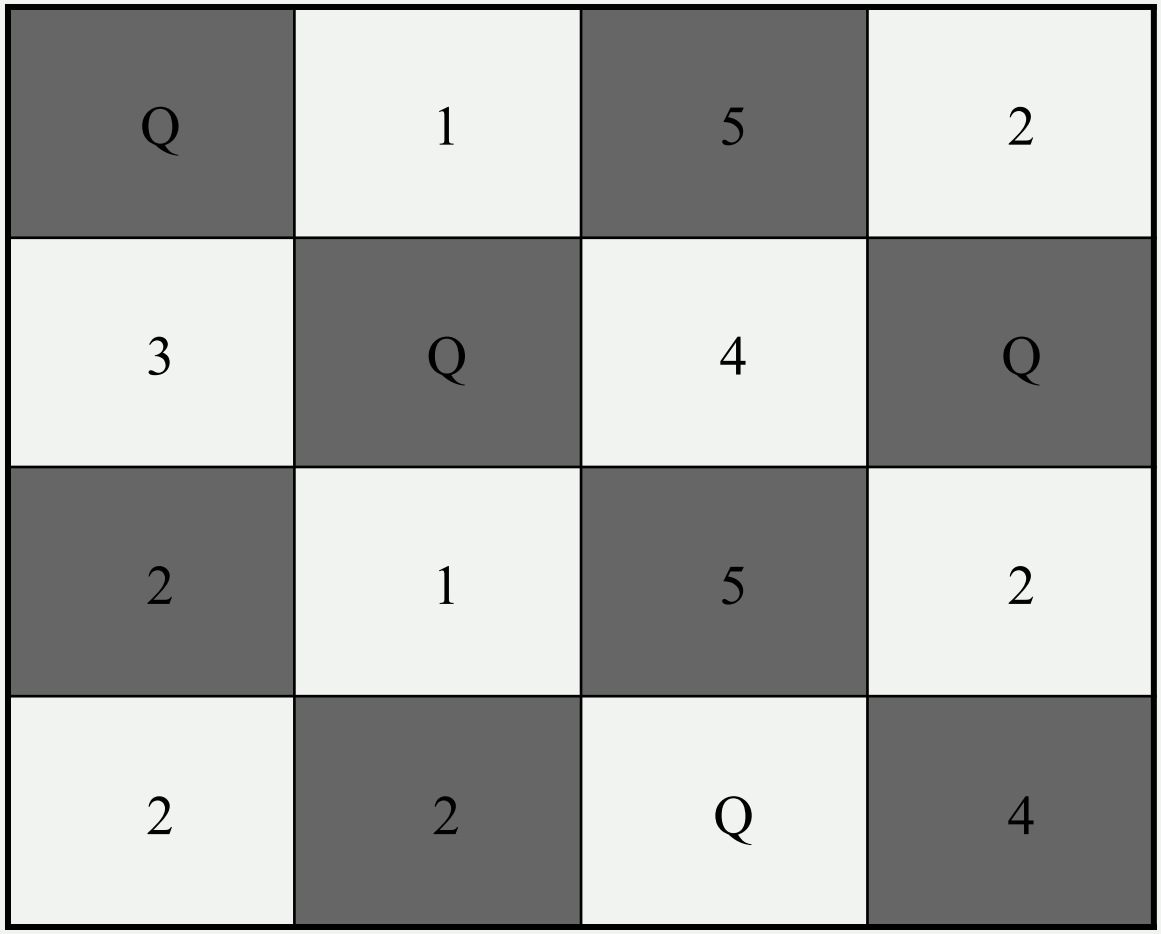
cost=5

N-Queens Hill Climbing – 2nd attempt



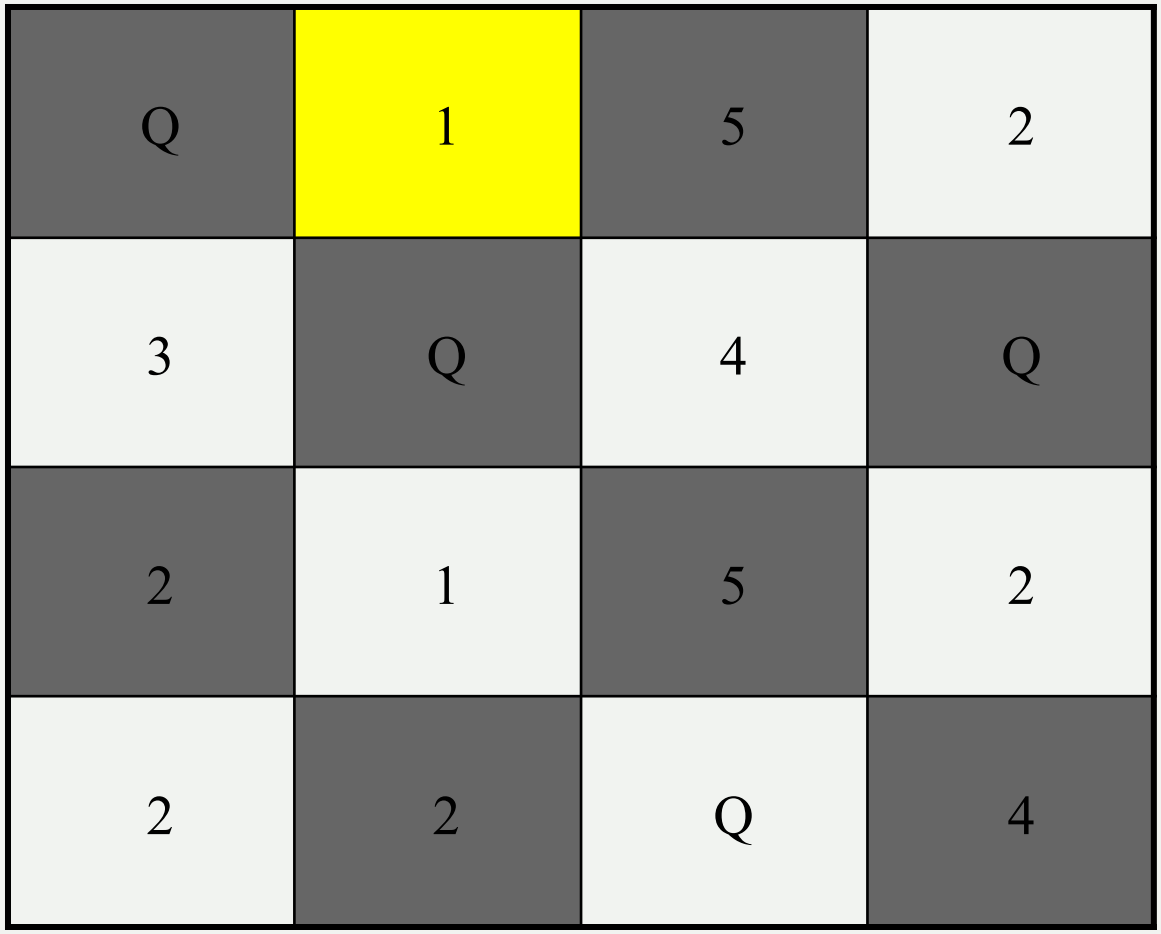
cost=5

N-Queens Hill Climbing – 2nd attempt



cost=2

N-Queens Hill Climbing – 2nd attempt



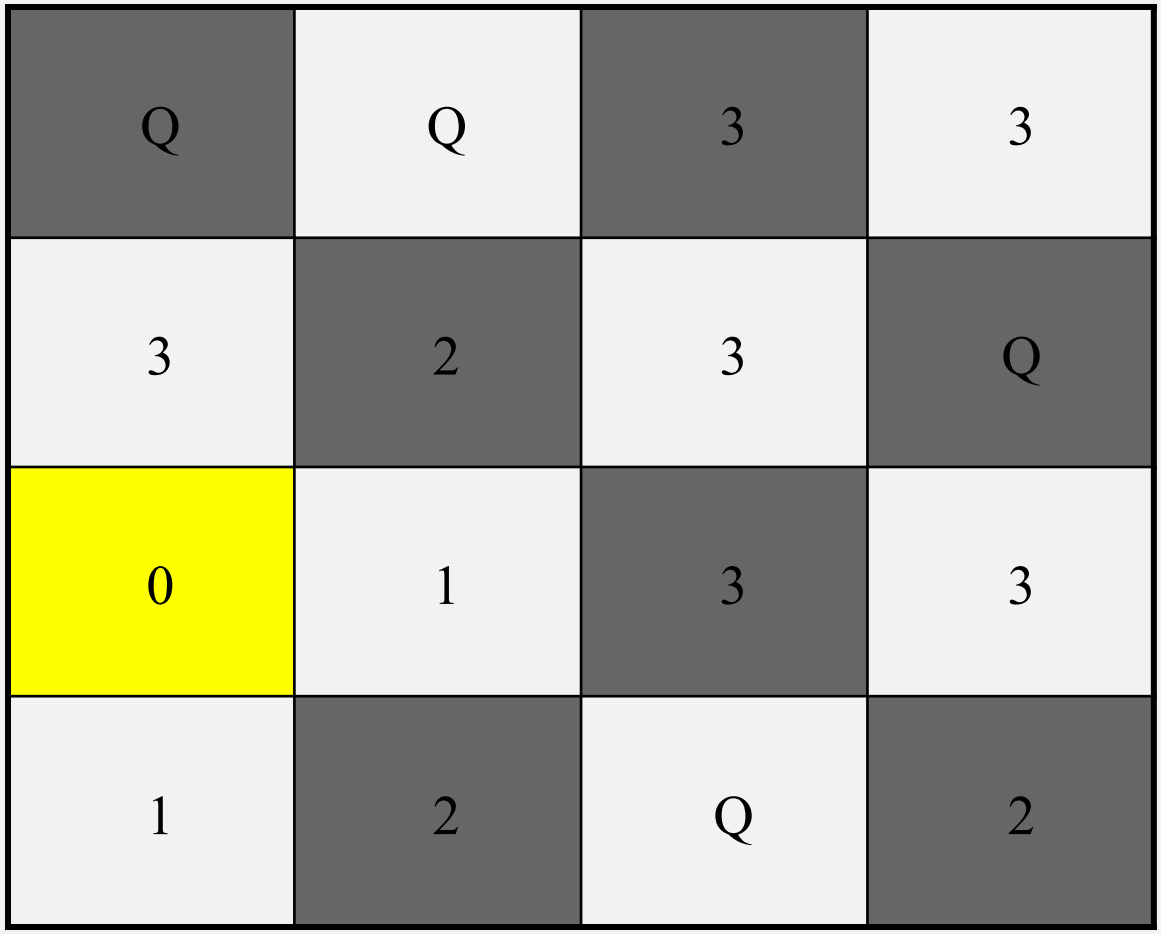
cost=2

N-Queens Hill Climbing – 2nd attempt

Q	Q	4	3
3	2	3	Q
0	1	3	3
1	2	Q	2

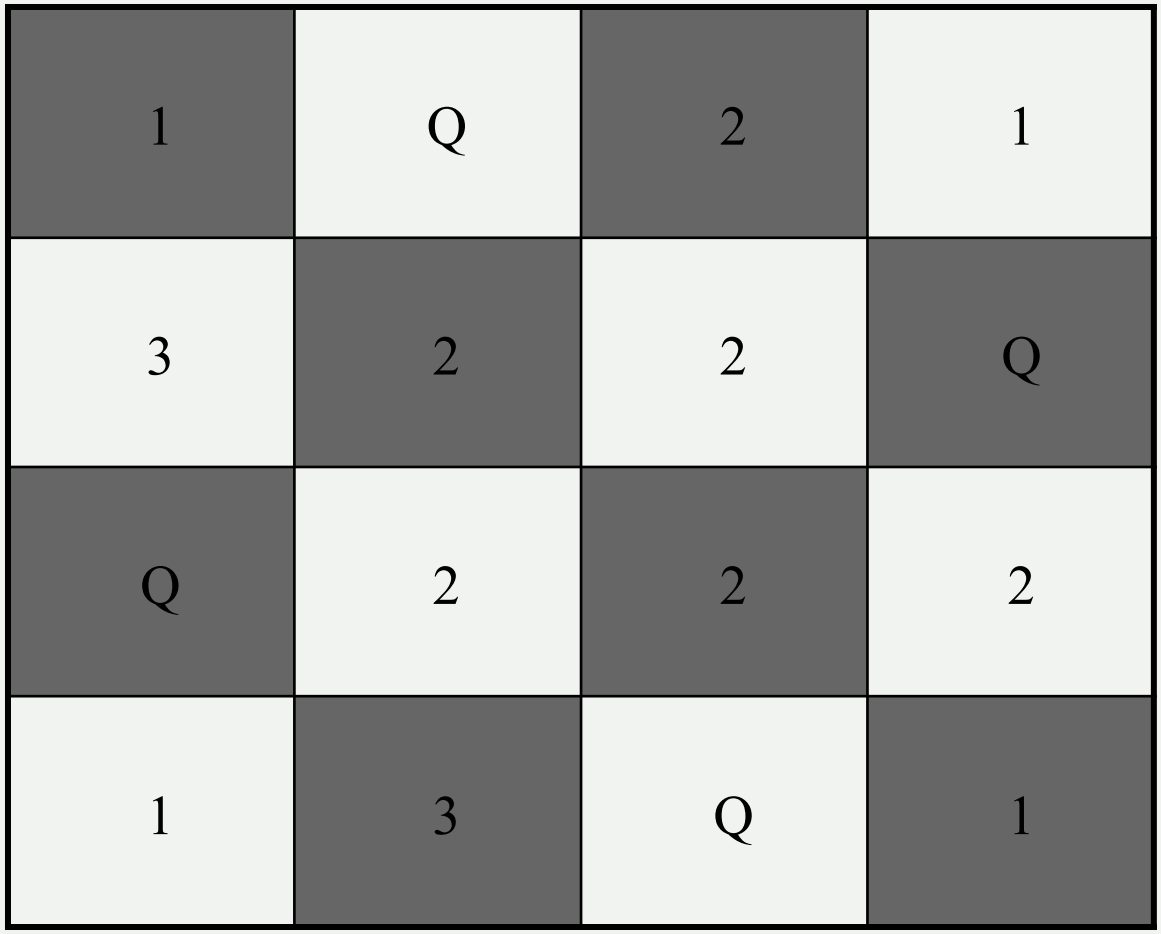
cost=1

N-Queens Hill Climbing – 2nd attempt



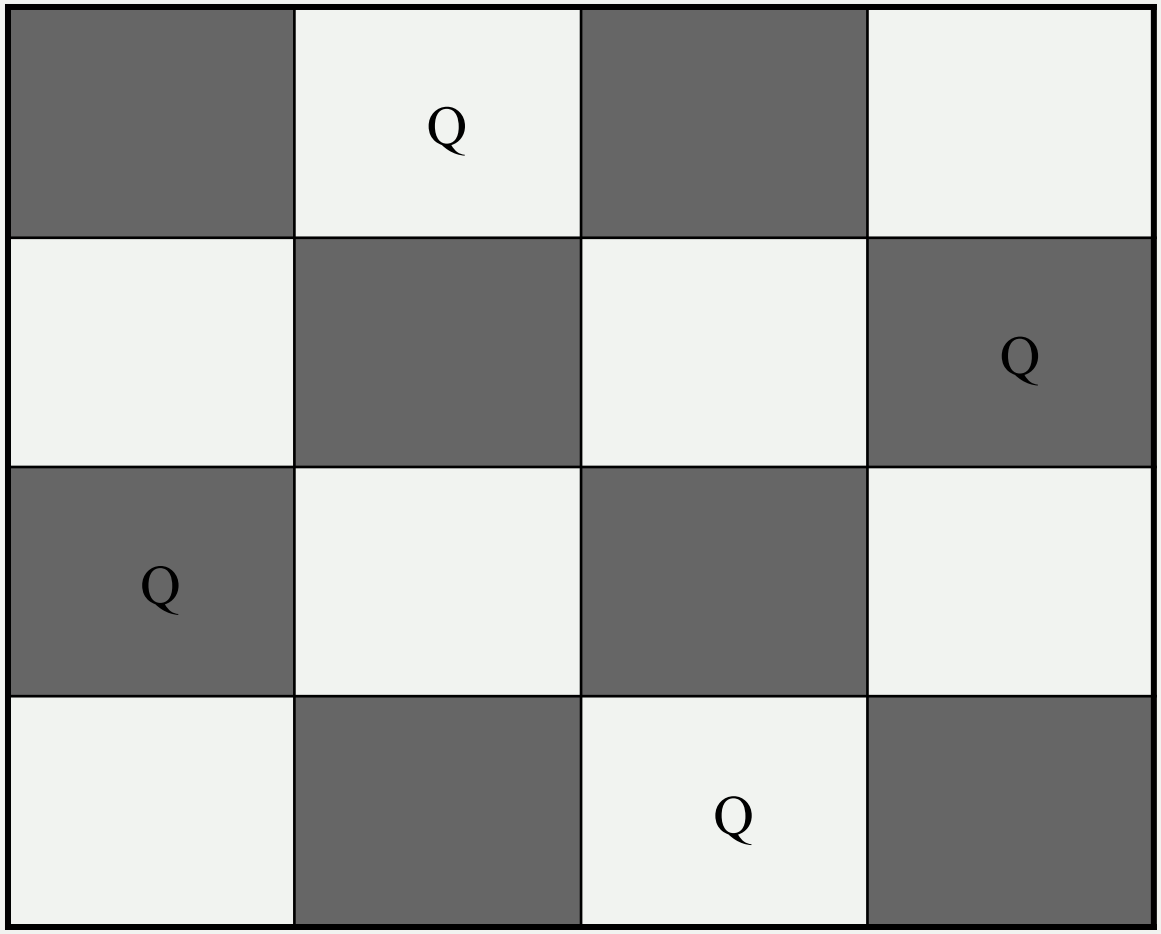
cost=1

N-Queens Hill Climbing – 2nd attempt



cost=0

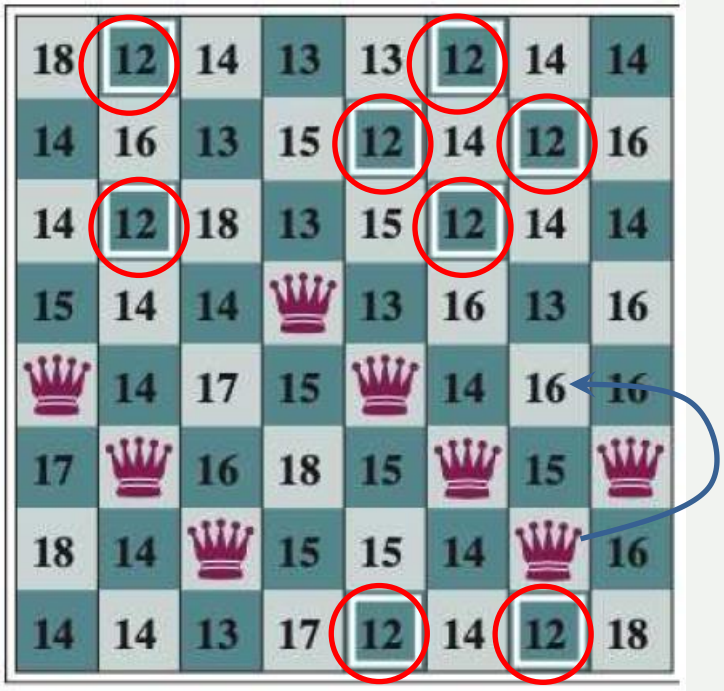
N-Queens Hill Climbing – 2nd attempt



cost=0

N-Queens problem

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum
  current ← *problem*.INITIAL
  while *true* **do**
    neighbor ← a highest-valued successor state of *current* // random choice // to break a tile
    if VALUE(*neighbor*) ≤ VALUE(*current*) **then return** *current*
    current ← *neighbor*



$h = 5 + 3 + 6 + 3 = 17$

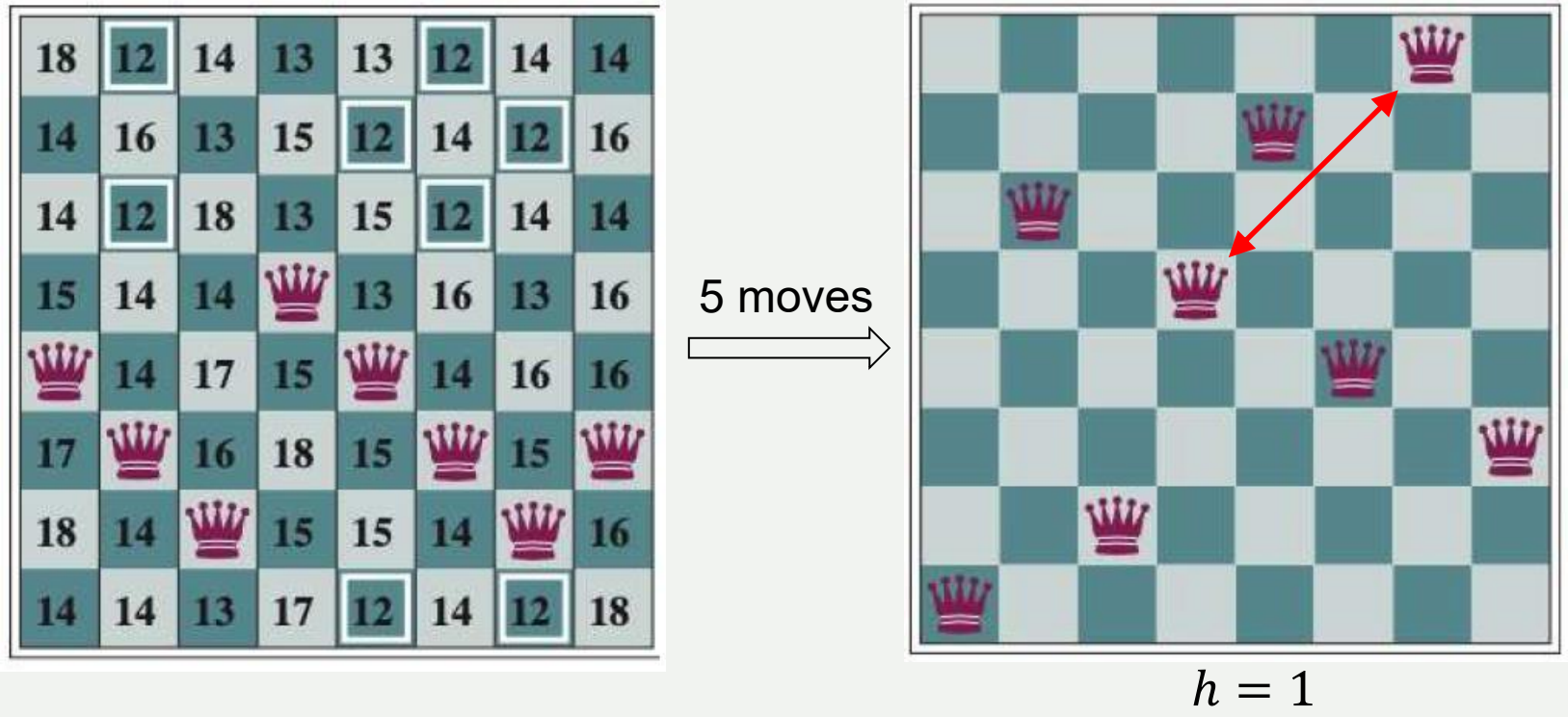
8-queens problem

- $h(s)$ = # pairs of queens attacking each other, directly or indirectly, in the state s.
- Successor is a state generated from relocating a queen in the same column.
- $h(s) = 12$ for the best successor.

  8-way tie!.

  Hill climbing randomly picks one.

Efficiency?

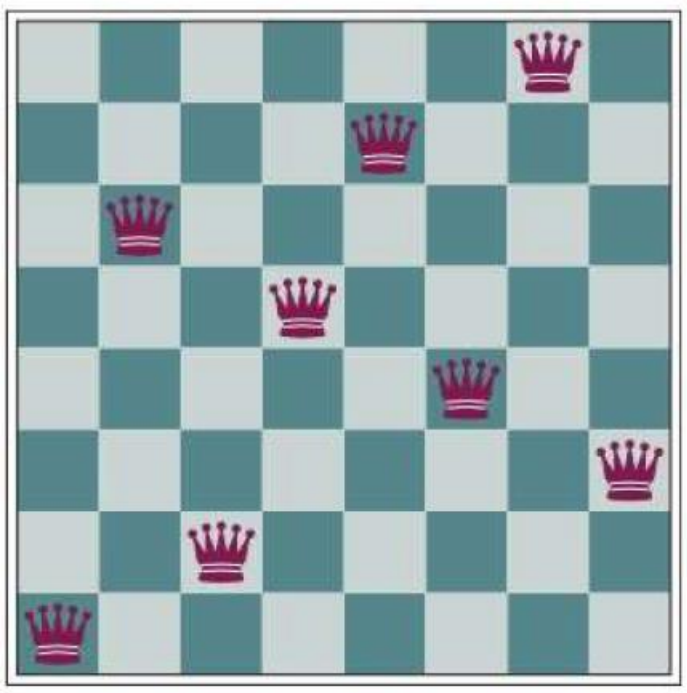


- Hill climbing can make rapid progress toward a solution.

Drawback of Hill Climbing

Hill climbing terminates when a peak is reached with no neighbor having a higher value.

♠ Local maximum →

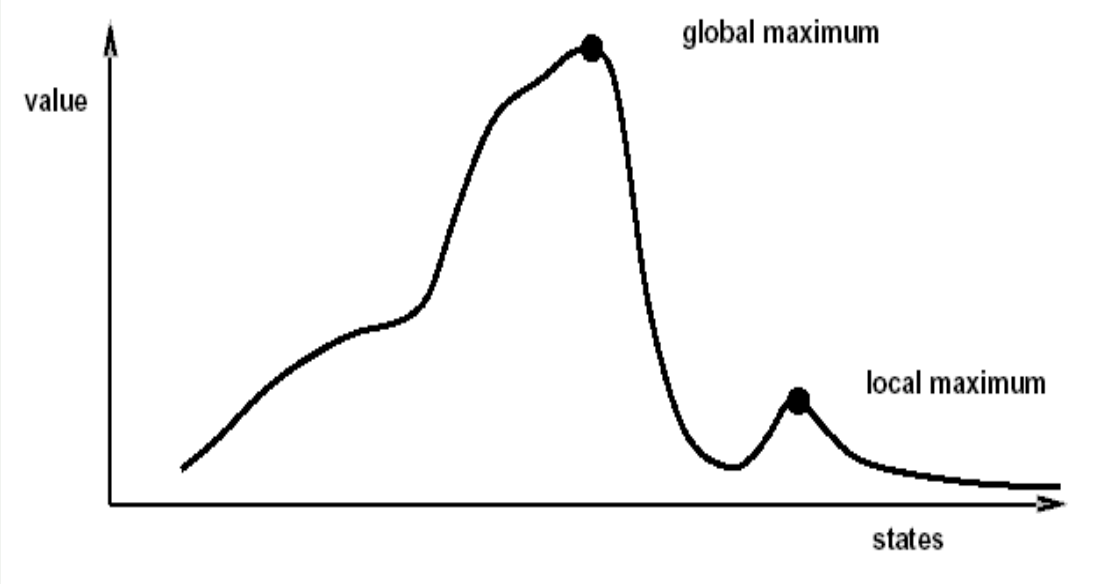


Every move of one queen introduces more conflicts.

Not the global maximum.

Hill climbing in the vicinity of a local maximum will be drawn toward it and then get stuck there.

Gradient Ascent/Descent



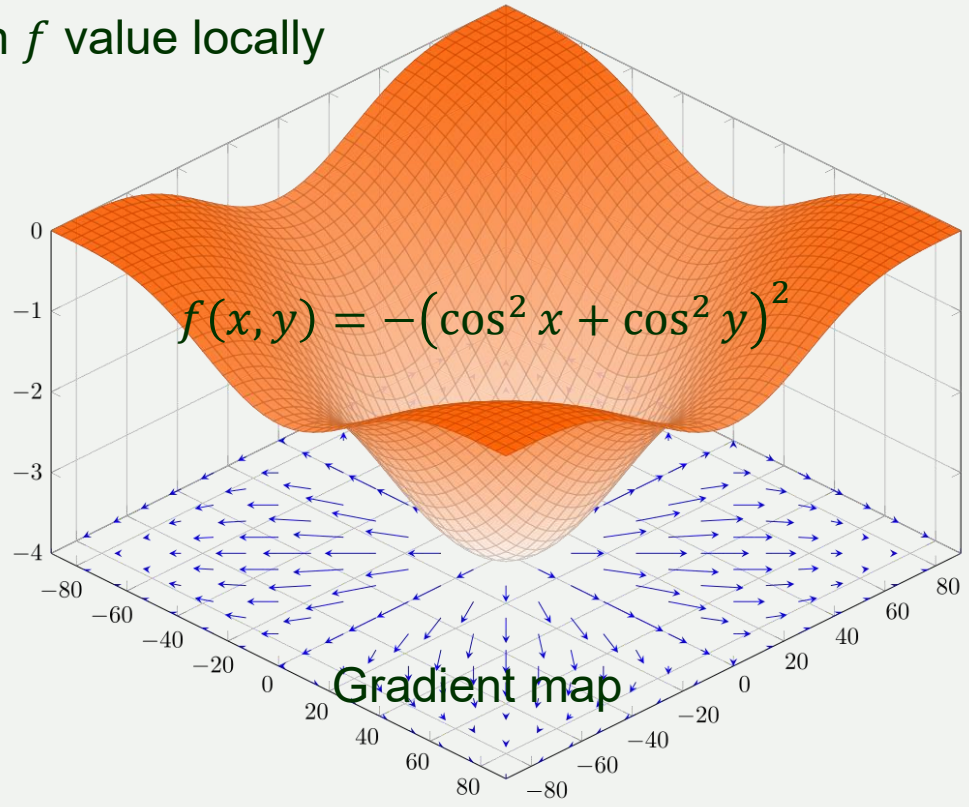
- A state as a point in n-dimensional space
  - One dimension for each variable
  - 4-queens problem – point in 4-space - e.g. (3, 1, 4, 2)
- Objective function
  - How “good” is that point in n-space?
- Try to find the max (or min) in that space
  - Can get stuck: randomness and restarts can help

Gradient-Based Method

Use the *gradient*: $\nabla f(x_1, \ldots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right)$

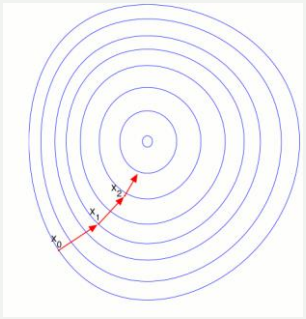
direction of fastest increase in f value locally

$-\nabla f$: direction of fastest decrease in f value locally



* From https://en.wikipedia.org/wiki/Gradient

Gradient Descent



- Assume we have some cost-function: $C(x_1,\ldots,x_n)$
  and we want minimize over continuous variables X1,X2,..,Xn

1. Compute the *gradient* : $\frac{\partial}{\partial x_i} C(x_1,\ldots,x_n) \qquad \forall i$

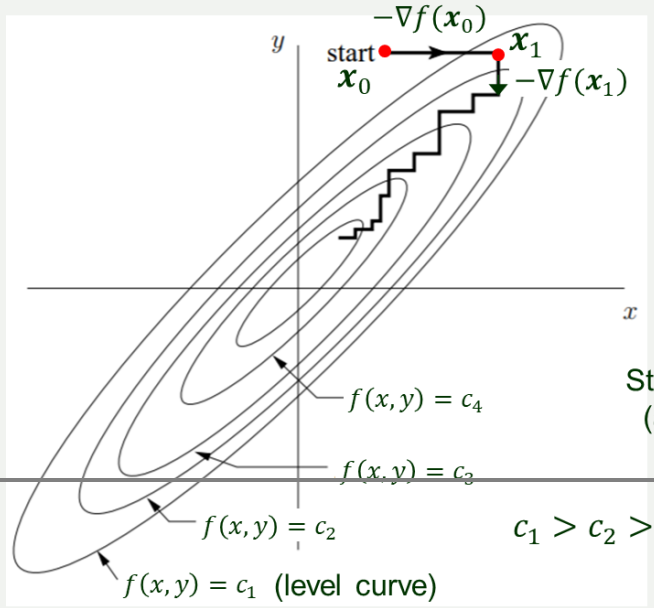
2. Take a small step downhill in the direction of the gradient:

$$x_i \rightarrow x'_i = x_i - \lambda \frac{\partial}{\partial x_i} C(x_1,\ldots,x_n) \qquad \forall i$$

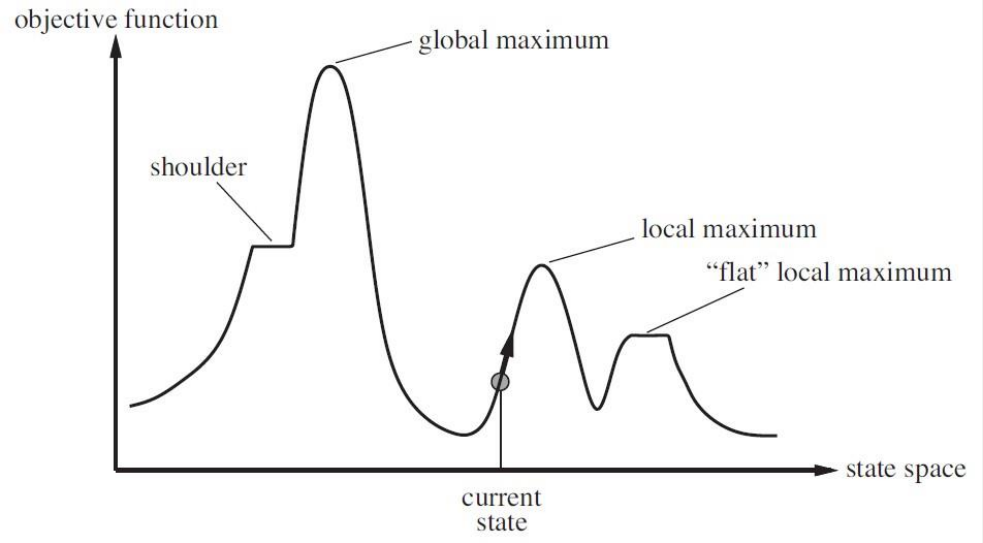
3. Check if $C(x_1,..,x'_i,..,x_n) < C(x_1,..,x_i,..,x_n)$

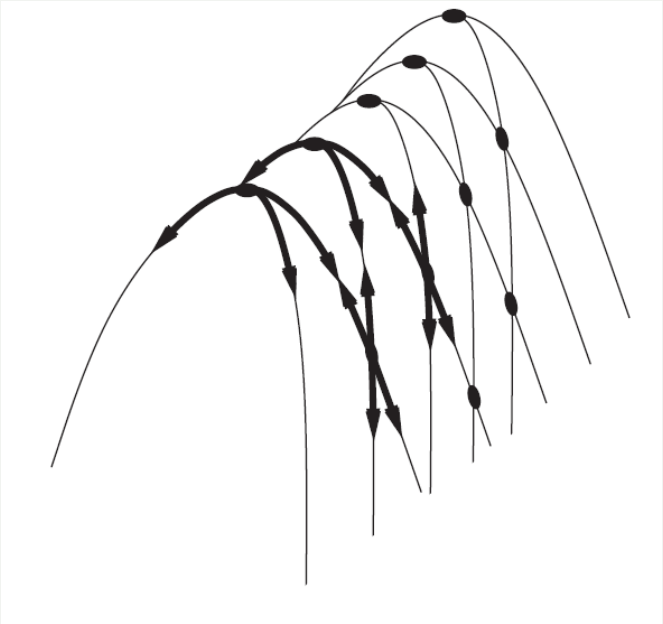
4. If true then accept move, if not reject.

5. Repeat.



Getting Stuck





- Local maxima: top of a smaller hill
- Plateau: flat local maximum
- Shoulder: flat place, can still go up on one side
- Ridge: local maxima not connected – the worst type

How to improve hill-climbing?

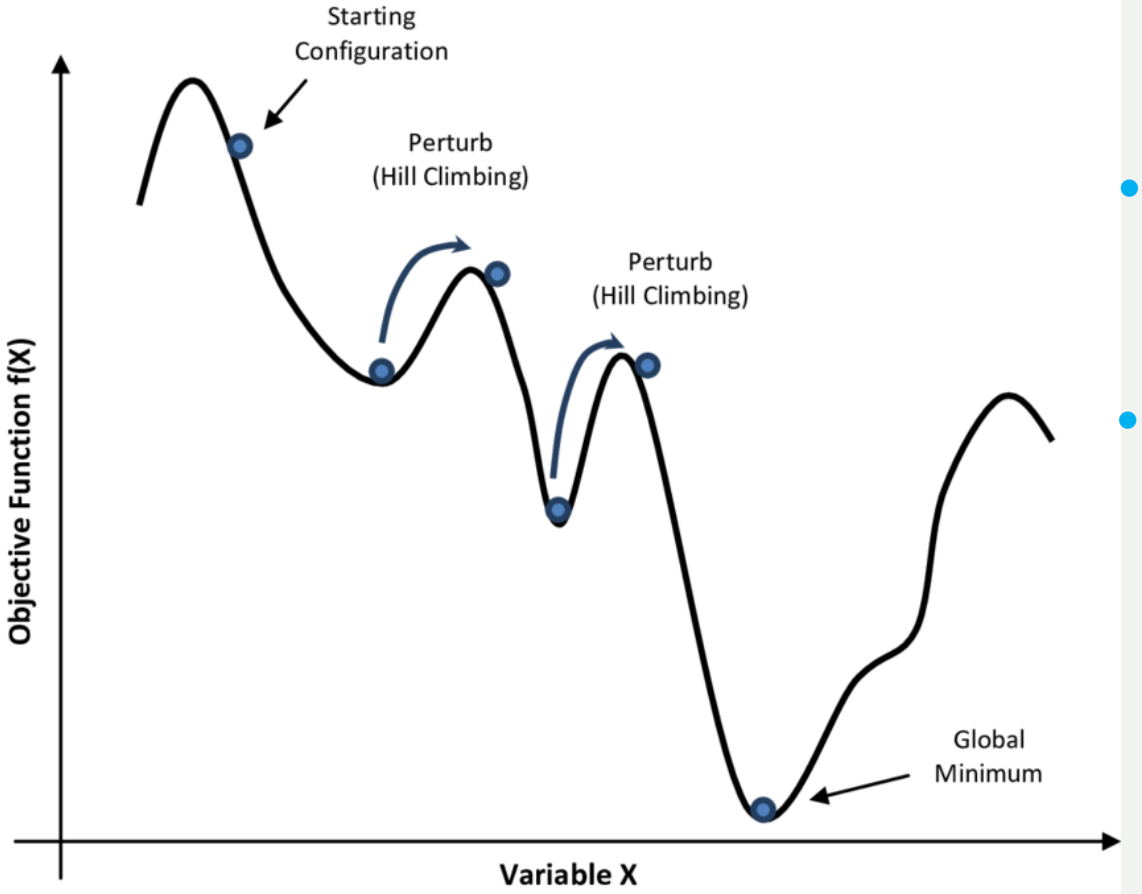
- Hill-climbing is already very useful for high-dimensional, continuous variable problems
- To leave a plateau – take sideway moves
- Randomly restart when stuck
- Randomly move “downhill” with some probability

3/5. Simulated Annealing

- Rolling a ball on a bumpy surface, trying to find the lowest point
  - Momentum can move it out of local minima
  - Shake the surface to move it out of local minima
- Instead of taking “best” move, take random move
  - If new state is better, accept move
  - If new state is worse, accept with some probability < 1
  - Probability of accepting worse point is determined by “temperature”

➔ escape local maxima by allowing some "bad" moves but gradually decrease their frequency

Annealing: Heat a metal to a high temperature and then gradually cool it, allowing the material to reach a low-energy crystalline state so it is hardened.



- Start by shaking hard (i.e., at high temperature).
- Gradually reduce the intensity of shaking (i.e., lower the temperature).

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current ← problem.INITIAL
  for t = 1 to ∞ do
    T ← schedule(t)
    if T = 0 then return current // solution
    next ← a randomly selected successor of current
    ΔE ← VALUE(current) – VALUE(next)
    if ΔE > 0 then current ← next   // the next state is better; choose it. .
    else current ← next only with probability e^{ΔE/T}   // ΔE ≤ 0
```

Minimization

Temperature → T

Badness Is $-\Delta E$.

$\lim_{t \to \infty} T = 0$

- Accept the next state if it is an improvement ($\Delta E > 0$).
- Otherwise, accept it with a probability that decreases exponentially
  - as the badness $-\Delta E$ of the move increases ($\frac{\Delta E}{T} < 0$ decreases), and
  - as the "temperature" goes down ($\frac{\Delta E}{T} < 0$ decreases).

Bad moves are more tolerated at the start when T is high and become less likely as T decreases.

- Escape local minima by allowing bad moves.

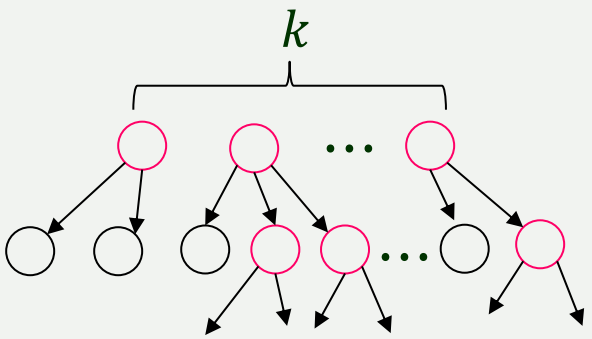
Properties of simulated annealing search

- One can prove: If *T* decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1 (however, this may take VERY long)

- Widely used in VLSI layout, airline scheduling, etc.

4/5. Local beam search

Keep track of k states rather than one.



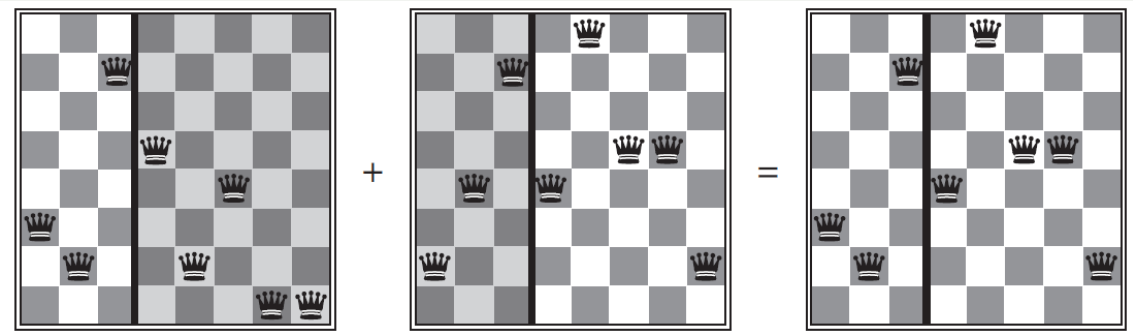
1. Start with k randomly generated states.
2. Generate all their successors.
3. Stop if any successor is a goal.
4. Otherwise, keep the k best successors and go back to step 2.

♣ May suffer from a lack of diversity among the k states.

Solution: stochastic beam search which chooses successors with probabilities proportional to their values.

5/5. Genetic algorithm

- Idea: mimic evolution in biology
- Similar to stochastic beam search
  - Keep k best successors with some randomness
  - Survival of the fittest: decide which descendants to keep by an objective (=fitness) function
  - Use two states, instead of one, to generate new states



Genetic algorithm

- A successor state is generated by combining two parent states
- Start with *k* randomly generated states (population)
- A state is represented as a string over a finite alphabet (often a string of 0s and 1s)
- Evaluation function (fitness function). Higher values for better states.
- Produce the next generation of states by selection, crossover, and mutation

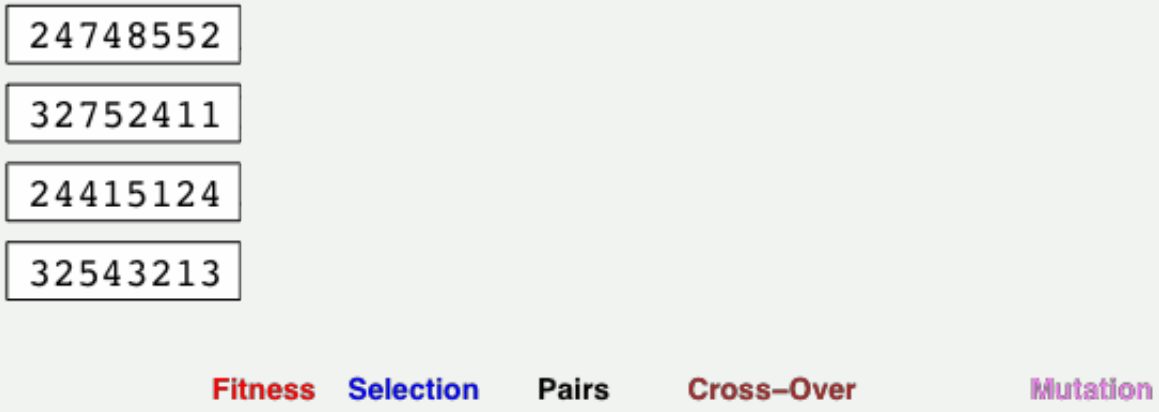
How genetic algorithms work

- Encode a state by a string:
  - 8-queens: (8 positions per queen) * (3bit per position) = 24 bits
  - One string = one state = an individual
- Population: k individuals
- New generation from previous generation
  - Crossover: exchange parts of strings from two parents
  - Mutation: change bits randomly for exploration
  - Selection: remove unfit individuals

How genetic algorithms work

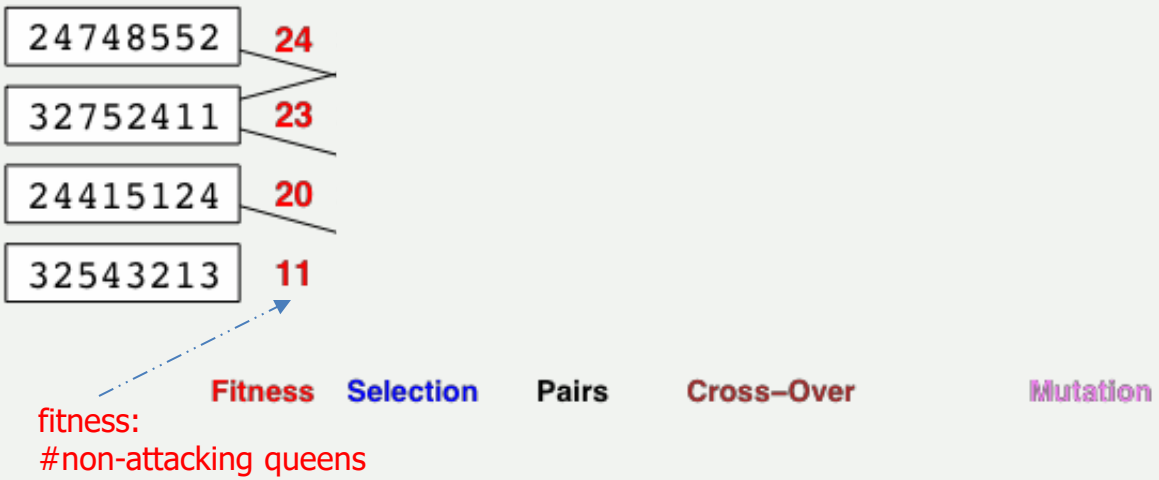
- Fitness function
  - How good is any particular individual (=state)?
  - Same as objective function
    - 8-queens: How many pairs of non-attacking queens?
    - Circuit layout: How much wire/solder saved?
    - Scheduling: How much time saved?
  - Fitness determines the probability that the genes of an individual will be carried on into the next generation

Example of a genetic algorithm



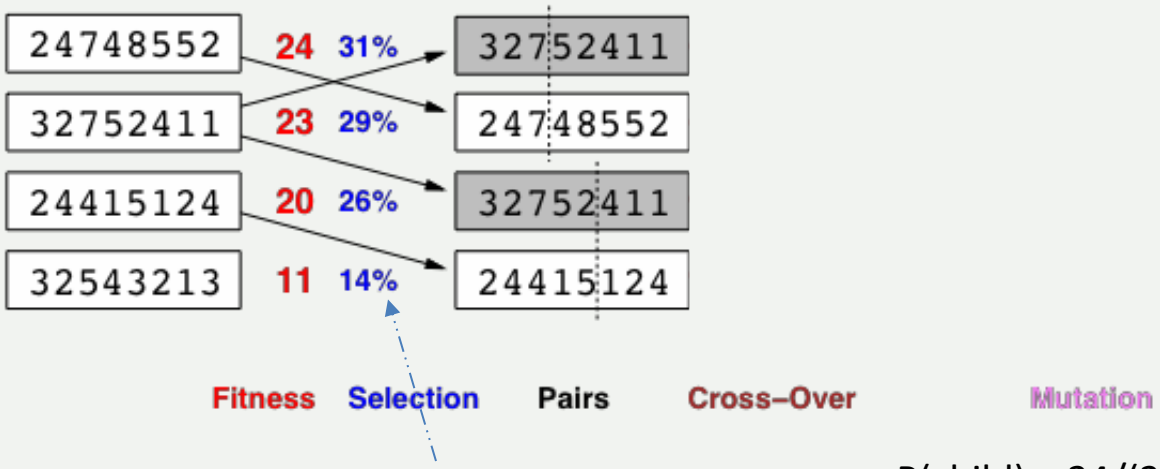
- Initial population
  - k randomly generated strings

Example of a genetic algorithm



- Fitness function
  - How good is each individual?
  - Probability of being selected for reproduction is proportional to the fitness function
    - Normalize the fitness values so that they sum to 100%

Example of a genetic algorithm

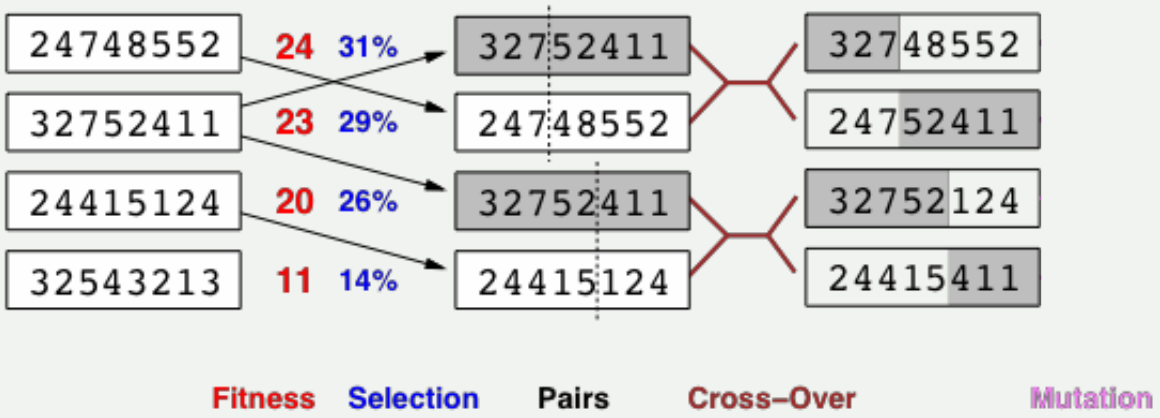


probability of being regenerated in next generation

P(child) = 24/(24+23+20+11) = 31%
P(child) = 23/(24+23+20+11) = 29% etc

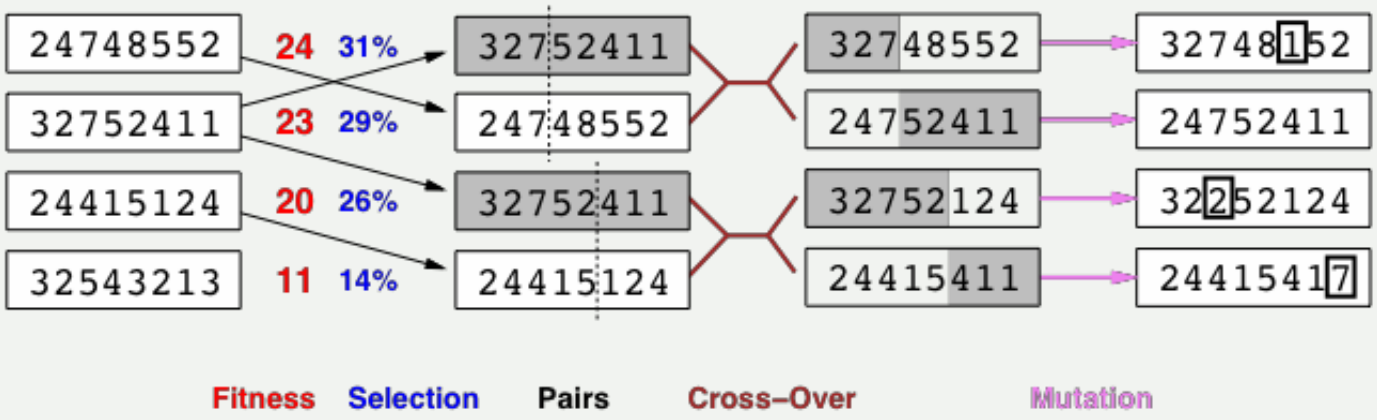
- Selection
  - Choose individuals randomly with the probabilities
  - Selected individual will be one of the parents
    - There is no male/female distinction
  - An individual can produce more than one child
    - It is not a monogamy

Example of a genetic algorithm



- Crossover
  - Make new children from a pair of parents
    - Choose a cut-point and swap halves
  - Why mix the genes in chunks and not randomly?
    - Assumption: adjacent digits/bits encode a solution to a *sub-problem*
    - Children share the solution to sub-problems with parents

Example of a genetic algorithm



- Mutation
  - Change bits randomly with a low probability
  - Like simulated annealing, help to get out of local minima

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights ← WEIGHTED-BY(population, fitness)
    population2 ← empty list
    for i = 1 to SIZE(population) do
      parent1, parent2 ← WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child ← REPRODUCE(parent1, parent2)
      if (small random probability) then child ← MUTATE(child)
      add child to population2
    population ← population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n ← LENGTH(parent1)
  c ← random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Applications of GA

- Complex structured problems

  Circuit layout, job-shop scheduling

- Evolving the architecture of deep neural networks
- Finding bugs of hardware
- Molecular structure optimization
- Image processing.
- Learning robots, etc.